

Modernizing UPS API Services with Devin

Java 8 → 21 • Spring Boot 2.7 → 3.5 • proven on UPS's own Track API example

What we did, what it proves, and what it is worth to UPS
Summary + ROI

The starting point: UPS's public API examples

Devin × UPS

github.com/UPS-API/java-api-examples — track, shipping, rating, pickup, timeInTransit, ...

- **Java 8 (2014).** `maven.compiler.source/target` = 1.8 in every module.
- **Spring Boot 2.7.4 / Spring 5.3.18.** Both past open-source support — no free CVE patches.
- **~240 lines of hand-pinned versions per pom.** CVE band-aids for commons-io, plexus, snakeyaml, jackson, ... that drift out of date.
- **JUnit 4, javax.* generated clients.** Blocks the Jakarta EE 9+/Boot 3 ecosystem.
- **Does not compile on JDK 21.** Lombok 1.18.24 crashes the JDK 21 compiler (NoSuchFieldError: JCTree\$JCImport.qualid).

Why this matters to UPS

- Track and Ship are revenue-and-SLA systems, not toy CRUD apps.
- Unsupported frameworks = open CVE exposure a product-security lead has to carry.
- UPS job postings already ask for Java 17/21 + Spring Boot on OpenShift — the codebase lags the hiring bar.
- Same pom shape repeats across ~10 example modules and many internal Spring services.

What Devin did: one session, one merged PR

Devin × UPS

Fork → ingest → reproduce failure → plan → upgrade → build/test → PR → Devin Review → merge

1 Ingest	Cloned the fork, mapped the track module: OAuth client-credentials + Track GET via generated RestTemplate clients.
2 Reproduce	Proved the baseline: builds on JDK 17, fails to compile on JDK 21.
3 Upgrade	Boot parent 2.7.4 → 3.5.16, java.version 21, OpenAPI Generator 7.25 with useJakartaEe=true, JaCoCo 0.8.12; deleted ~240 pinned lines the Boot BOM now manages.
4 Harden tests	New offline test: OAuth POST → Bearer → Track GET → parsed response via MockRestServiceServer (no UPS credentials). Live CIE tests gated behind UPS_CIE_TESTS=true.
5 Verify & ship	mvn clean package on JDK 21: BUILD SUCCESS, 1 passed / 3 skipped, zero javax.annotation in generated code. PR with before/after evidence; Devin Review: no issues.

Hand-written application code untouched — the entire change is pom.xml + test hardening (+85 / -243 lines).

Before / after: the evidence on the PR

Devin × UPS

Every number below is reproduced in the PR description and the recorded demo

	main (before)	PR #1 (after)
mvn clean package on JDK 17	passes	passes
mvn clean package on JDK 21	FAILS — Lombok vs JDK 21 javac	passes
Java / Spring Boot	8 / 2.7.4	21 / 3.5.16
spring-web	5.3.18 (EOL)	6.2.19
jackson-databind	2.14.0	2.21.4
snakeyaml	1.33	2.4
Generated client annotations	javax.annotation.*	jakarta.annotation.*
pom.xml	265 lines, ~20 plugin pin blocks	–240 lines; Boot BOM manages versions
Tests	3 live calls needing UPS CIE credentials	+ offline happy-path test; live tests opt-in

Devin Review on the PR: “No Issues Found”. Same recipe repeated on shipping (PR #5, 381 → 158 pom lines); next: rating, pickup, timeInTransit, ...

Why this lands for UPS

Devin × UPS

Security, scale, engineer time, and evidence

Retire the CVE exposure

Boot 2.7 / Spring 5.3 / Java 8 are unpatched in SLA-critical Track & Ship paths. Devin turns “we should upgrade” into a reviewed PR in a day.

Scale across the estate

~10 example modules and many internal Spring services share this pom shape. Prove the recipe once; Devin runs it in parallel — module N costs minutes of review.

Engineers review, not grind

Transitive versions, Jakarta namespace, generator flags, dead pins, Lombok/JaCoCo/JDK compatibility — the work seniors hate is what Devin does well.

Audit-ready evidence

Every PR ships the failing baseline, passing build, dependency deltas and credential-free tests — what a change board or product-security lead actually needs.

Hiring & platform

Java 21 + Boot 3 matches the roles UPS is hiring for and unlocks virtual threads, observability and native images on OpenShift.

A repeatable loop

Ingest → plan → code → build/test → PR → Devin Review → merge, with a persisted JDK 21 environment so each next run starts warm.

Proof point: Mercedes-Benz — COBOL modernization Devin × UPS

Same legacy-modernization pattern as UPS's Java 8 estate, on an older and harder stack

The use case

- **Who.** Mercedes-Benz global engineering — software behind R&D, logistics, cloud infrastructure and enterprise operations across the US, Europe and Asia.
- **The problem.** A legacy COBOL estate that had to be understood and modernized without compromising reliability, stability or security.
- **What Devin did.** In a four-week pilot, analyzed 200,000+ lines of COBOL and carried the modernization work that engineers had estimated at eight months.
- **Outcome.** Modernization time cut from ~8 months to 8 days. Mercedes-Benz then deployed the full Cognition suite (Devin, Windsurf, Devin for Terminal) across R&D and IT.
- **Why it maps to UPS.** Same shape: large, aging codebase; engineers review rather than rewrite; evidence-driven change in systems the business cannot afford to break.

200,000+

lines of COBOL analyzed by Devin in the pilot

8 mo → 8 days

estimated modernization time vs actual

Scale of deployment
One of the most extensive AI software-engineering deployments in the automotive industry — global R&D and IT teams on three continents.

Source: cognition.com/blog/mercedes-benz-cognition (Apr 2026)

Fork of UPS-API/java-api-examples • PR #1 merged:
github.com/matth4922-droid/UPS-java-api-examples/pull/1

Proof point: GE Aerospace — ~2× output

Devin × UPS

Revenue-and-safety-critical software, engineers directing and reviewing Devin end to end

What GE Aerospace gets from Devin

- **Nearly 2× output.** One GE Aerospace Software-as-a-Service team nearly doubled its engineering output after rolling out Devin.
- **More enhancements shipped.** Stated ambition: deliver more software enhancements to customers while raising the bar for quality — not trading one for the other.
- **End-to-end, not autocomplete.** Devin works across the lifecycle — writing code, then carrying it through testing, review and iteration.
- **Engineers stay in control.** GE engineers direct and review the work throughout; Devin adds capacity, the team keeps accountability.
- **Where it runs.** Software that optimizes routes, reduces fuel burn and monitors anomalies for airlines — GE engines power 3 of 4 commercial flights. High-stakes, like Track and Ship.

~2×

engineering output on one GE Aerospace SaaS team

3 of 4

commercial flights worldwide powered by GE Aerospace engines

The UPS parallel
Same operating model we propose here:
Devin does the upgrade grind and verification; UPS engineers plan, review and merge.

Source: GE Aerospace Software-as-a-Service and Cognition posts on LinkedIn (Aug 2026)

ROI — cost per service upgrade

Devin × UPS

Outside-in model; assumptions stated. Replace with UPS-internal rates to calibrate.

Approach	Effort per module	Rate assumption	Cost per module
In-house senior Java engineer	2-6 eng-weeks (dependency triage, Jakarta, test repair, review)	\$4-5k / week fully loaded	\$10k - \$30k
SI / contractors	Same effort + onboarding, knowledge walks out the door	\$150-250 / hr	\$25k - \$60k
Devin + engineer review	Hours of compute + 2-4 hrs human review & merge	~\$100 compute + reviewer time	\$0.5k - \$1.5k

90 - 95%

lower unit cost per service upgrade vs in-house

Days → hours

wall-clock per module; modules run in parallel

~\$15 - 25k

saved per service (midpoint vs in-house)

Effort ranges reflect a service of the track/shipping shape (generated clients, Spring RestTemplate, no web layer). Larger internal services skew toward the top of the range for humans; Devin's compute cost rises far less.

Devin cost is order-of-magnitude from this session (tens of ACUs at list pricing) plus reviewer time at the in-house rate.

ROI — across a 100-service portfolio

Devin × UPS

Scales linearly; UPS likely has more than 100 Spring services in scope

Approach	Cost (100 services)	Calendar time
In-house	\$1.5M - \$3M	12-18 months, sequential, competes with roadmap
SI / contractors	\$2.5M - \$6M	6-12 months, plus procurement
Devin + review	\$50k - \$150k	Weeks — modules run in parallel

Avoided cost (larger, harder to bound)

- **Breach / incident risk.** US-average breach cost \$9.4M (IBM 2024). Expected loss from the unpatched estate: ~\$0.5-2M per year — see next slide.
- **Commercial Java support.** Any Java 8 estate on Oracle support is billed per employee — material at UPS headcount. Retiring it removes the line item.
- **Platform efficiency.** Boot 3 / Java 21 virtual threads and native images lower OpenShift footprint for the same traffic.

\$1.5M - \$5M

direct labor savings per 100 services

4 - 8 eng-yrs

returned to revenue features per 100 services

The pitch line
“~\$15-25k saved per service, months to weeks, and the security exposure ends this quarter instead of next year.”

ROI — the cost of a breach

Devin × UPS

Expected-loss view of running Spring 5.3 / Boot 2.7 / Java 8 past end of support

Benchmark (IBM Cost of a Data Breach 2024)	Per incident
Global average	\$4.88M
United States average	\$9.36M
Mean time to identify + contain	258 days
Not included: SLA credits, shipper churn	UPS-internal figures

Expected loss / yr = annual breach probability × cost per breach

Scenario	Annual probability	Cost basis	Expected loss / yr
Conservative	5%	\$4.88M global	\$0.24M
Base	10%	\$9.36M US	\$0.94M
Adverse	20%	\$9.36M US	\$1.87M

Probabilities are placeholders for UPS product-security to replace with their own risk register. Benchmarks are per-incident averages, not UPS figures; a single Track/Ship incident is the whole number, not the expected value.

\$0.9M / yr

base-case expected loss avoided by retiring the EOL stack

~6-19×

base-case avoided loss vs. Devin cost of upgrading 100 services (\$50-150k)

Why the probability is non-zero
Spring Framework 5.3 left OSS support Aug 2024 and Boot 2.7 in Nov 2023: new CVEs get no free patch. Spring4Shell (CVE-2022-22965) hit exactly this line.

ROI — UPS cost and revenue outcomes

Devin × UPS

Cost is the proven pile of cash; revenue quality is why the AI program exists. Both depend on current APIs.

Cost — already quantified by UPS

- **~\$3.5B** Network Reconfiguration + Efficiency Reimagined savings in 2025.
- **~\$4.5B** expense removed via the Amazon glide-down (~2M pieces/day of low-yield volume out).
- **~\$3B** program benefits targeted for FY26; ~\$1.2B booked in 1H26.
- **28% cheaper/piece** in automated buildings; automation at ~68.5% of U.S. volume; RFID removes hundreds of millions of manual scans/yr.

Revenue — better mix after the reset

- **~\$91.2B** FY26 revenue guidance, raised from ~\$89.7B; adj. operating profit ~\$8.65B.
- **\$22.8B, +7.6% YoY** Q2 2026 revenue after a planned volume cut; U.S. Domestic adj. OP +20%+.
- **Mix** SMB 34.5% of U.S. volume (+250 bps); healthcare logistics >\$3B/qtr; DAP >\$1.4B, B2B DAP +34%.
- **Stickiness** Control towers, RFID visibility, Happy Returns and AI brokerage keep SMB / healthcare / industrial shippers from dual-sourcing lanes.

June 2026 AI initiative	If it works	P&L effect
Tracking + AI assistants on >98% of CS requests by YE26	Fewer calls, faster claims	Cost first, then retention
RFID + AI package-level tracking, digital twin (10-min refresh)	Fewer manual scans, self-healing network	Cost (labor, idle assets)
Agentic control towers; AI brokerage / customs (97% first-day entry)	Exception mgmt for big shippers; predictable landed cost	Revenue (enterprise, cross-border, SMB)

The June AI program does not replace the \$3B-\$4.5B cost takeout; it converts it into higher-quality revenue and a network that still works when volume returns. That only holds if Track, Ship, Rate, brokerage and the digital twin stay current — a Java 8 / Boot 2.7 estate under those APIs is a business problem, not a tidy-up.

Sources: UPS June 18, 2026 AI release; UPS Q2 2026 earnings / FY26 guidance. Savings are booked by UPS to network redesign and mix — not attributed to AI.

Next steps

Devin × UPS

From one proven module to a portfolio program

Done

shipping module — PR #5

Same recipe on the next revenue module: 381 → 158 pom lines, 10 live CIE tests gated, new offline OAuth → Ship happy-path test, green on JDK 21. main still fails today — the live ‘before’ for the meeting.

Done

Close the loop with CI — PR #4

GitHub Actions builds upgraded modules on JDK 21 on every PR, so each Devin PR ends in a visible green check.

Next

Pilot on one internal UPS Spring service

Same before/after evidence format, UPS-internal rates plugged into the ROI model, inside UPS’s repo and review process.

Then

Portfolio run

Inventory Spring 2.x / Java 8 services; Devin runs the upgrade in parallel, engineers review, product-security gets the evidence pack.